

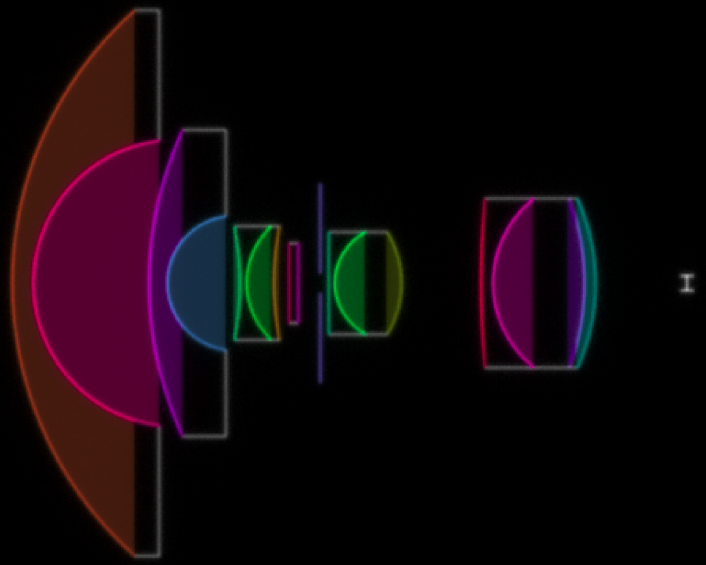
Implementing a Full Spectral Lens Sim

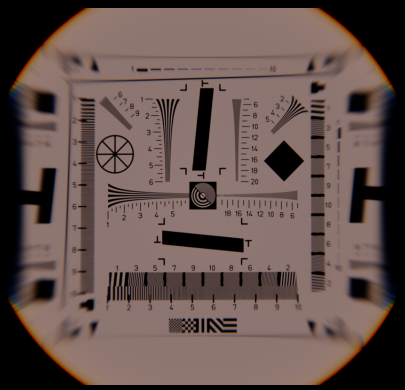
October 30, 2026

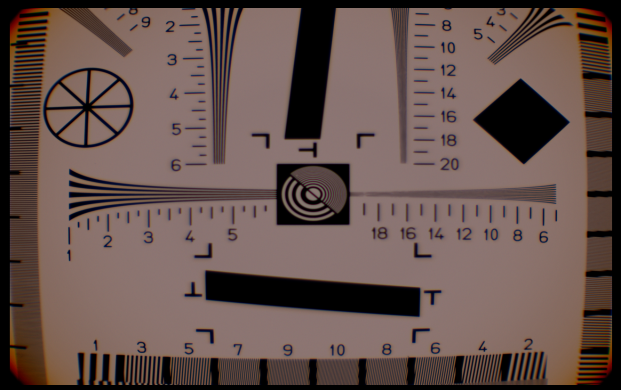
Jon Baker (jb)
twitter: @jbaker_graphics / bsky: @jbakergraphics
project blog: https://jbaker.graphics/

What to Expect

- I've recently implemented a GPU-based system in Vulkan for resolving the image through a lens system using ray optics (loosely based on PBRTv3's Realistic Camera) which is capable of modelling a lot of the aberration and distortion of specific lens designs, and want to share some of my experience
- I'm going to show the output of a number of debug tools which I implemented during the process, and quickly explain some relevant optics fundamentals
- I'm going to talk about the specification of a lens system, how to read the data in a lens prescription table, and how to use it to build a system like this

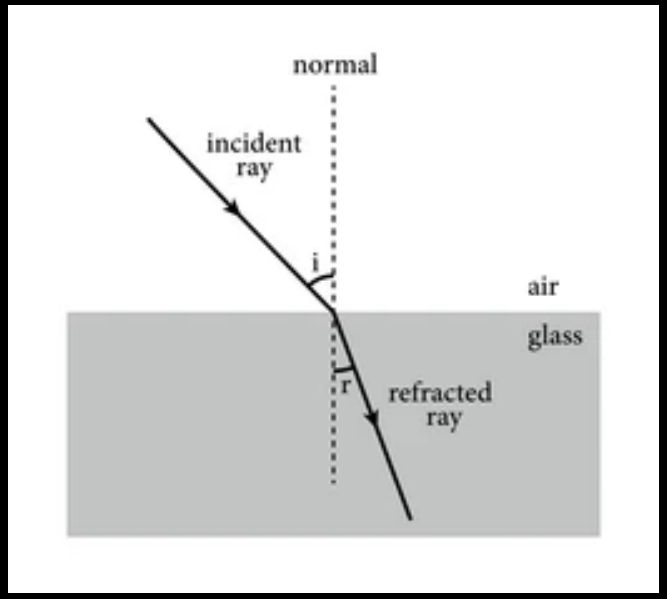






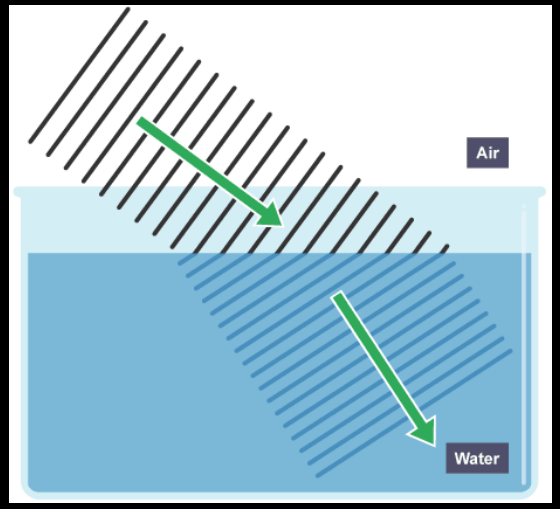
Snell's Law - Refraction

- Refraction happens when we transition between media of different index of refraction (IoR), N_1 and N_2
  - This is a unitless property of the material, and we are just taking a ratio of the two
  - The ratio of these two properties is inverted, when you are going the other direction (air into glass vs glass into air)
- We can calculate the result of the refraction in ray optics using the GLSL refract() function
  - Parameters are the surface normal, the incident ray, and the ratio of N_1 and N_2, referred to as eta
  - Result is the refracted ray, or a zero vector if the refraction math fails



Snell's Law - Wave Interpretation

- The IoR property actually describes the propagation speed of light in that medium, relative to how fast it travels in a vacuum
- This gives rise to another interpretation based on wave optics... pulses of incident energy arrive at some rate, and the bending ocurrs as a result, because they share phase at the boundary
- The specifics here are a little bit out of scope, but we can say that this is a phenomenon that changes with the rate of the pulses ariving at the interface
  - "Rate" here refers to the frequency of the incident light - this property corresponds to its color, as it is proportionally related to the wavelength of the light by the constant speed of light C
  - This variation of IoR with respect to frequency is referred to as "Dispersion", and it is a key concept in optical systems - it is the same phenomenon that is depicted on the cover of Dark Side of the Moon, and which gives rise to rainbows when it ocurrs inside of water droplets in the atmosphere





Abbe Number and Cauchy Coefficients

- It's important to have an expression for this behavior, if we want to be able to use it for a real application like an optical system - relevant for describing different types of glass, fluids, or atmospheric gasses
- Framework for describing this property came from Ernst Abbe in the mid-to-late 1800s, defining the index and Abbe number of optical glass
  - These are defined at reference wavelengths of light - specific emission bands from luminous gas discharge provide a frequency reference that can be used to test a sample of a given material
- The Cauchy Eqauation is from somewhat earlier, in 1830
  - The most useful expression for our use here is the two term form

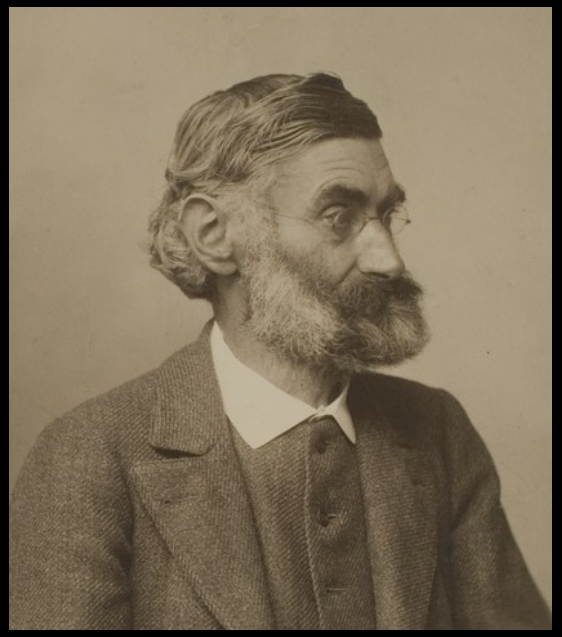
$$n(\lambda) = A + B/(\lambda^2)$$

- It is possible to derive the Cauchy Equation coefficients A and B corresponding to a given IoR and Abbe number, by this relation given by LuxCoreRender (using the d-, F-, and C-wavelength definitions):

$$B = \frac{n_d - 1}{v_d} * \frac{F^2 C^2}{C^2 - F^2} = \frac{n_d - 1}{v_d} * 0.52$$

$$A = n_d - B * 2.897 \quad or \quad A = n_e - B * 3.354$$

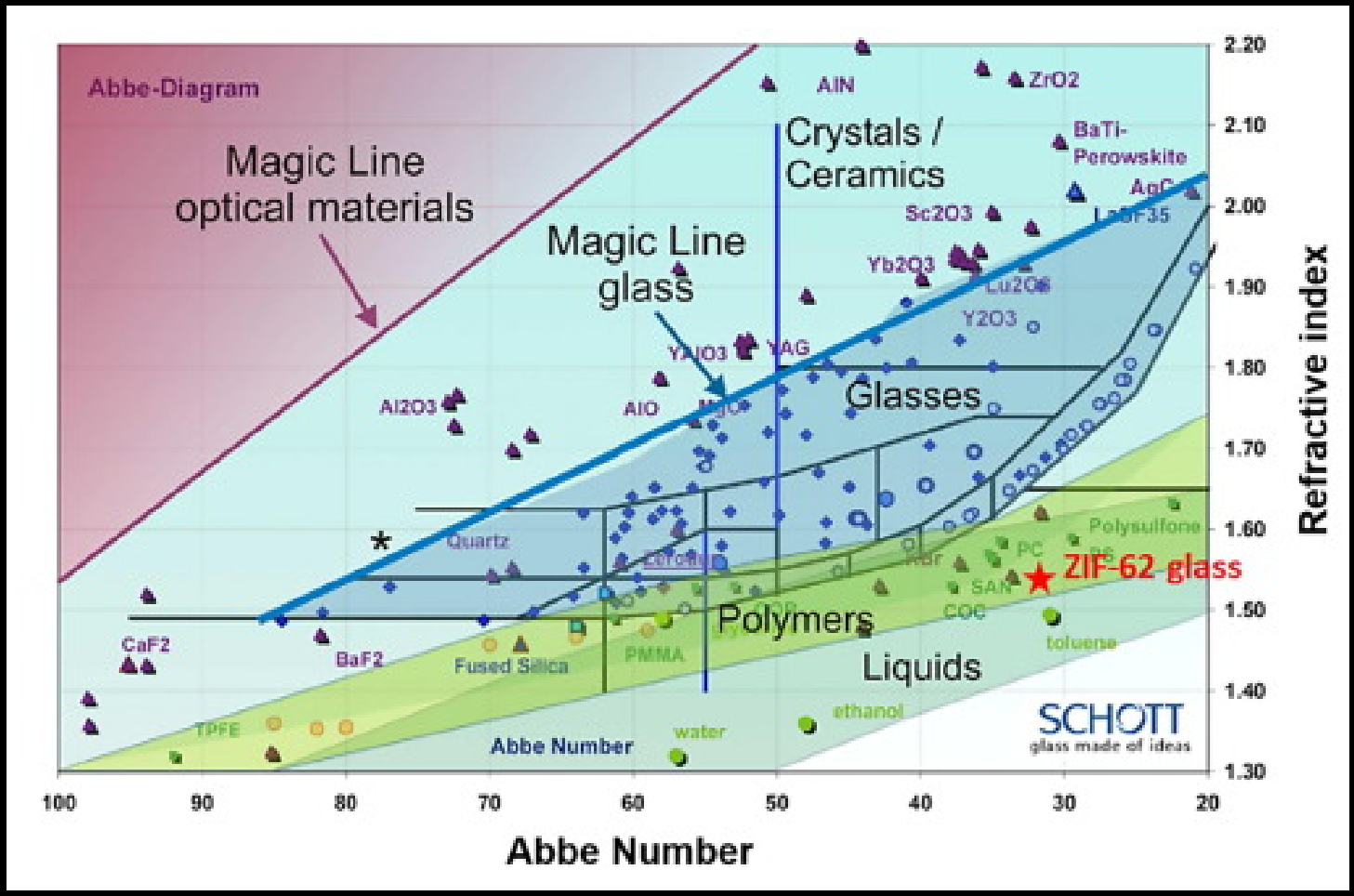
- They go on to explain that the difference between the two definitions is less than 0.05% over the visible spectrum of light, and that the difference between using Cauchy and the more math-heavy Sellmeier Equation is similarly negligible



Ernst Abbe (1840-1905)

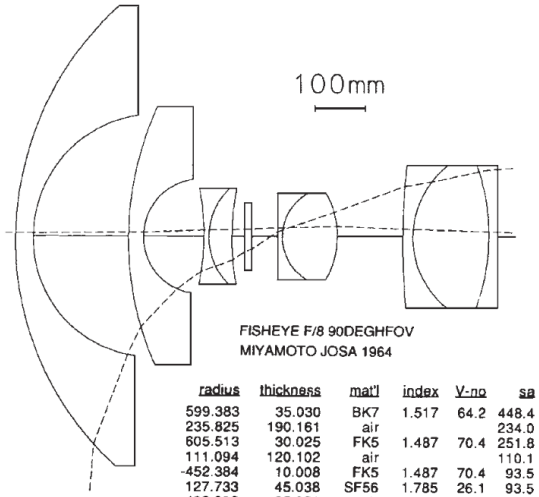
Abbe Diagram - The Glass Veil

- Different chemical formulations are used to control the refractive index and dispersion of engineered glasses
- You can visualize these properties on a 2D plot, which is called an Abbe Diagram, or the Glass Veil



Lens Prescriptions

- The term is probably more familiar from the opthamology field, the same is term is used here to describe a specification of a lens system
- This is an example fisheye design from Warren J. Smith's book Modern Optical Engineering
- Each line item is "an interface" in the system
- The parameters can be put in two categories
  - Material Properties
    - Index (IoR)
    - Abbe number (often labeled V_{no})
    - Usually a code for a type of glass that fulfills this requirement, or "air" for air gaps
  - Geometric Properties
    - Radius of curvature: this is element curvature, specifying convex, planar, or concave surfaces
    - Thickness: this is the distance from this interface to the next
    - Semiaperture: easy to confuse the abbreviation with with saggita, but this is equal to half the diameter of the element, what we see here as height

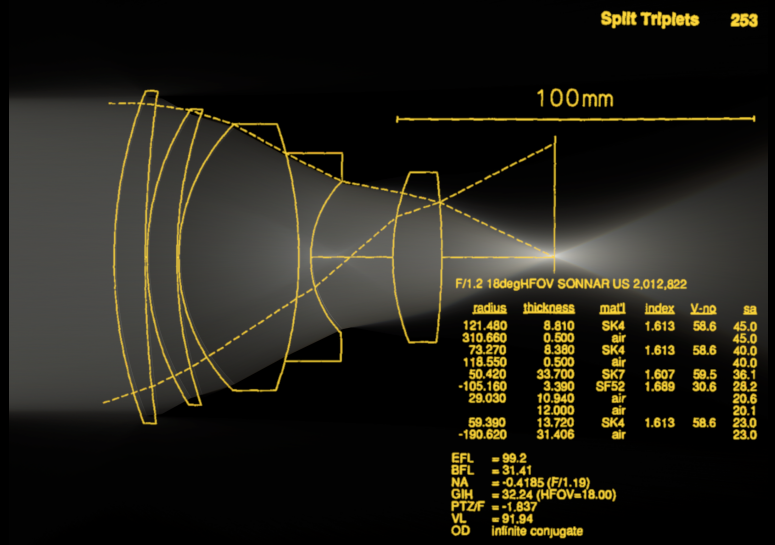


FISHEYE F/8 90DEGHFOV
MIYAMOTO JOSA 1964

radius	thickness	mat'l	index	V-no	sa
599.383	35.030	BK7	1.517	64.2	448.4
235.825	190.161	air			234.0
605.513	30.025	FK5	1.487	70.4	251.8
111.094	120.102	air			110.1
-452.384	10.008	FK5	1.487	70.4	93.5
127.733	45.038	SF56	1.785	26.1	93.5
462.892	25.021	air			93.5
	15.013	K3	1.518	59.0	65.4
	36.281	air			65.5
	13.762	air			15.8
38507.649	10.008	SF56	1.785	26.1	84.1
95.081	110.093	LAF2	1.744	44.7	84.1
-162.638	130.110	air			84.1
1376.167	20.017	SF56	1.785	26.1	139.0
177.275	150.127	BSF52	1.702	41.0	139.0
-400.339	18.766	BASF6	1.668	41.9	139.0
-337.536	150.119	air			139.0

Processing and Visualizing Lens Data

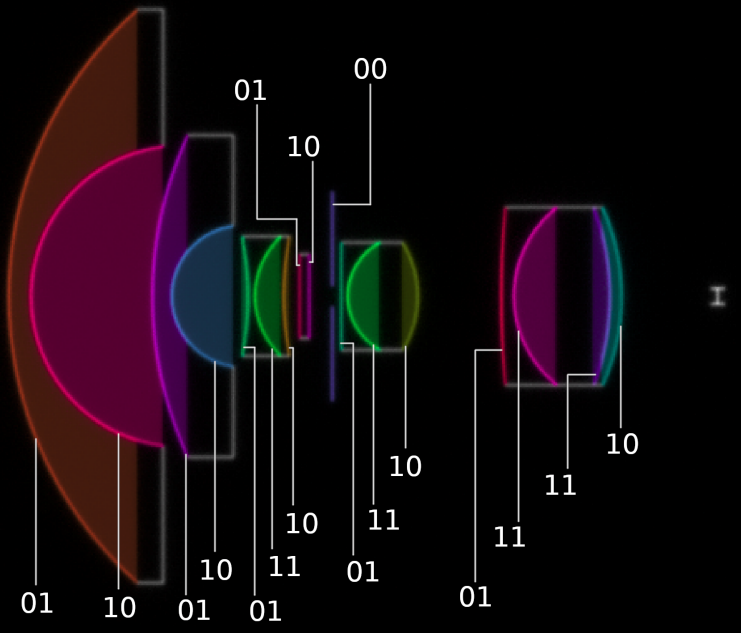
- Sectioned views like this are taking advantage of the radial symmetry of the system
- A lens system consists of one or more elements, in one or more groups - these are higher level organization structures which can be interpreted from the prescription
  - An element is a piece of glass that can be manufactured, as a building block of a system like this - the surfaces can be ground into concave, planar, or convex profiles, with spherical or aspherical curvature
  - A group is one or more elements grouped within the system - this can include things like cemented doublets and triplets, which are simple elements which are glued together with a material with properties such that it acts like a glass-glass interface
- "Optical Space"/"Camera Space" here is defined with Z going left-to-right, the film plane centered at the origin, offset backwards along the Z-axis by the total system thickness - the center of the front element is located at the origin
- We provide as much information in 2D as 3D, in this sectioned drawing, again radial symmetry



Showing agreement between my 2D pathtracer and the reference diagram in Modern Optical Engineering

Segmenting into Lens Elements

- Iterating through the table, we're assigning boolean space occupancy to each interface in the table
  - 0 means "air", 1 means "glass"
- We can assign a 2-bit code to each interface, which gives us 4 possible values
  - 00: air/air represents an aperture stop
  - 01: air/glass is the front side of an element
  - 11: glass/glass is between cemented glass elements
  - 10: glass/air is the back side of an element
- Why it matters? Building a system string like this, we can draw a better diagram now, and add cylinder geometry for the outer edge of these elements if desired
  - It's also potentially useful information at runtime, to control the refraction/masking logic when tracing a ray through the system

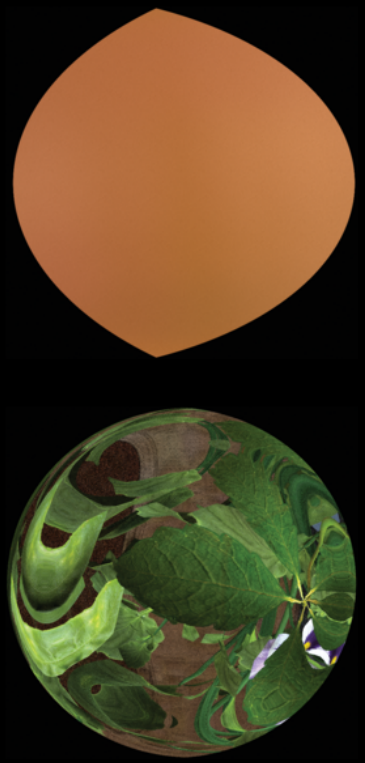


Tracing a Ray through the System

- What is a "valid ray"? We're using a somewhat simplified model
  - Starts at the film plane, with an associated wavelength, on the far right of our diagrams
  - Visits each element, in order, performing the correct wavelength-dependent refraction using the Cauchy coefficients of the materials on either side of the interface
    - The fact that there are no reflections is a key simplification for speed and it means that this system is not currently able to directly resolve lens flares at time of writing
    - I'm interested in exploring this further, in the style of one of my prior experimental forward pathtracer projects called Newton, where primary rays are emitted from light sources and would only contribute to the image if they ultimately intersect a simulated film plane
  - Passes all aperture tests, both explicit aperture stops (e.g. a bladed iris) and those defined implicitly by the semiaperture of each element
  - Arrives at the font surface of the front element, and is then ready to be traced against the scene

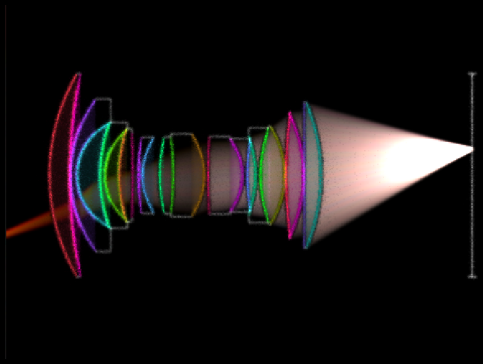
Pupil Sampling - Motivation

- Viewed from the film plane, a lens system has what's called an exit pupil - this is the effective aperture of the system, which admits scene radiance
- In order to resolve the correct image at the film plane, you need to integrate all the radiance at the pupil - this is equivalently the incident radiance at the film plane (for our purposes)
- This graphic, from Physically Based Ray Tracing v3's Realistic Cameras chapter, shows a view of the system pupil from two distinct points on the film plane
  - Somewhat counterintuitively, the one which is solid orange color is "in focus", as the incident radiance is uniform - we integrate, and get orange for this point on the film
  - The green one where the pupil sees distinct leaves will need to integrate varying frequency image contents - when we perform the integration, it constitutes a blurring, and we see this as an "out of focus" portion of the image
- We need an efficient way to sample this pupil radiance - when we do this integration for all points on the film, the result is the lens image that we want to resolve



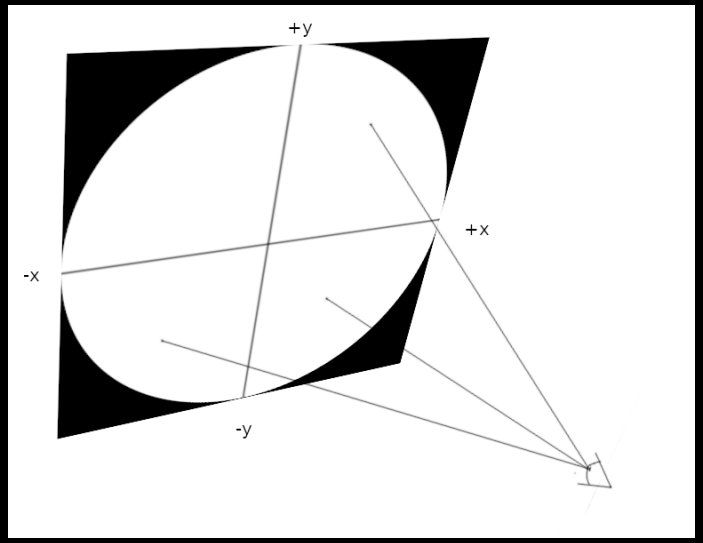
Establishing the Exit Pupil

- At render time, starting from the film plane, we need a way to generate rays that will pass through the exit pupil of the lens
- In the image to the right, this debug tool is drawing all ray paths starting at the film plane on the right and traced as far as they continue through the lens system - those rays arriving at the left hand side are considered "valid", and have made it through the system... the point is that we want to note how small a subset this is, of the total rays drawn
- Some sampling strategies, motivating our approach:
  - Hemisphere Sampling - generate random directions in the hemisphere normal to the film plane... this can be <1% efficiency, with very few rays making it the exit pupil... we can do better
  - Rear Element Sampling - generating random directions based on the vector from the flim plane to random points generated on the rear-most element of the system... this is more efficient than hemisphere sampling, and it is a much more efficient starting point
  - Pupil Sampling - if we can, for example, put a bounding box around the exit pupil, and generate uniform samples in that box... it's a good starting point for sampling the image, and the rate of successful rays generated this way is approximately the ratio of the areas of a circle inside a square, $\frac{\pi}{4}$
  - Importance Sampling of Pupil Radiance - this is still in the works in my pathtracer, Phoenix, but if we can accumulate some LUT of the pupil's radiance distribution, it will be able to resolve both a more efficient and a much more correct result... more about this in the discussion section at the end



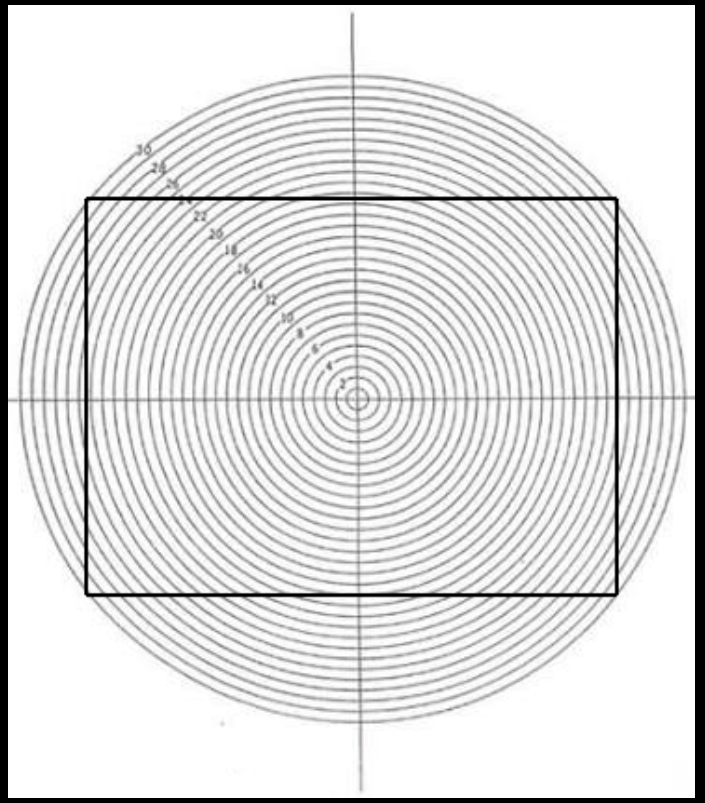
“Normalized Rear Element Coordinates”

- How do we define that bounding box? We need to define a space to operate in - inspired by LCRS’ project Lensform
- Why is rear element sampling valid as a starting point?
  - In order to arrive at the film, you must necessarily pass through the rear element of the system
  - In a light-sealed system (like most cameras), this is the aperture through which all light incident at the film plane must pass
  - Another way to say this - we will never recieve any light from any direction besides one which originates inside this rear element aperture
- This defines our problem... we can place a plane at the Z-axis location of the rear element, and on that plane, define a normalized space that spans -1 to 1 across the rear element aperture - I call this “Normalized Rear Element Coordinates”
- Great! Now we have a simple, scale-invariant 2D space to work in (we don’t care about changes to the lens parameters, if we need to convert to points in 3D space, we’ll just reference the current parameters for the rear element of the system)



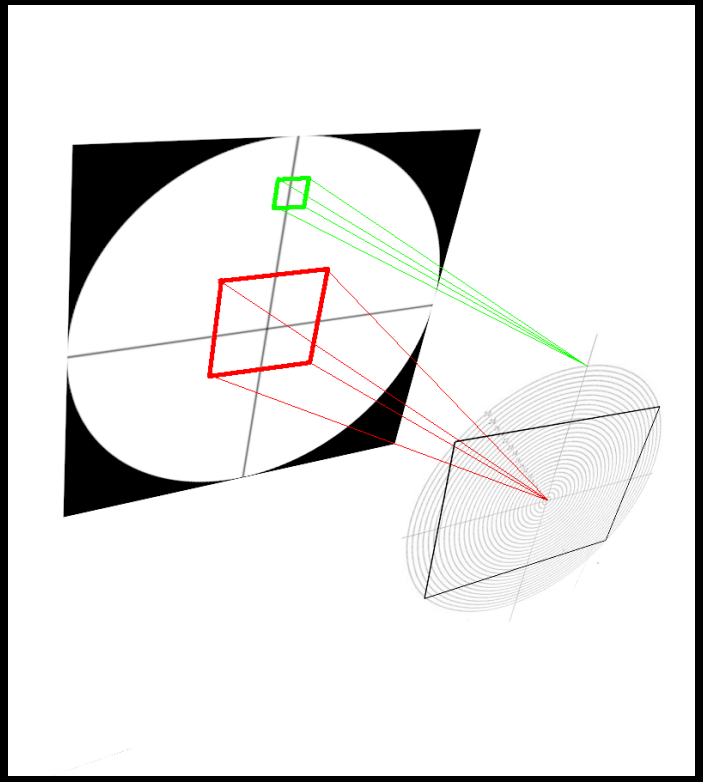
Creating Sampling Rays

- At this point we've defined everything we need to guide the pupil sampling, it's just putting it together
- We want to establish a bounding box for the system pupil in Normalized Rear Element Coords, and then use that bounding box at runtime to guide our generation of sample rays
  - Unfortunately, this set of bounds is not uniform for the entire film plane
  - We are lucky, though, in particular, because of the radial symmetry of the system that we mentioned earlier
- Ultimately what we need is a 1D LUT containing sampling information, which we index into using the distance from the center of the film plane
- Each LUT entry needs to be precomputed, upon any change to the lens system, film size, or focus offset, to store min and max on X and Y in our sampling space that can generate valid rays... I'll discuss consequences of the uniform sampling scheme at the end
- The LUT resolution is allocated with the first entry applying to 0 distance from the center, to entry N at the corner distance (this is the maximum possible displacement on the image)



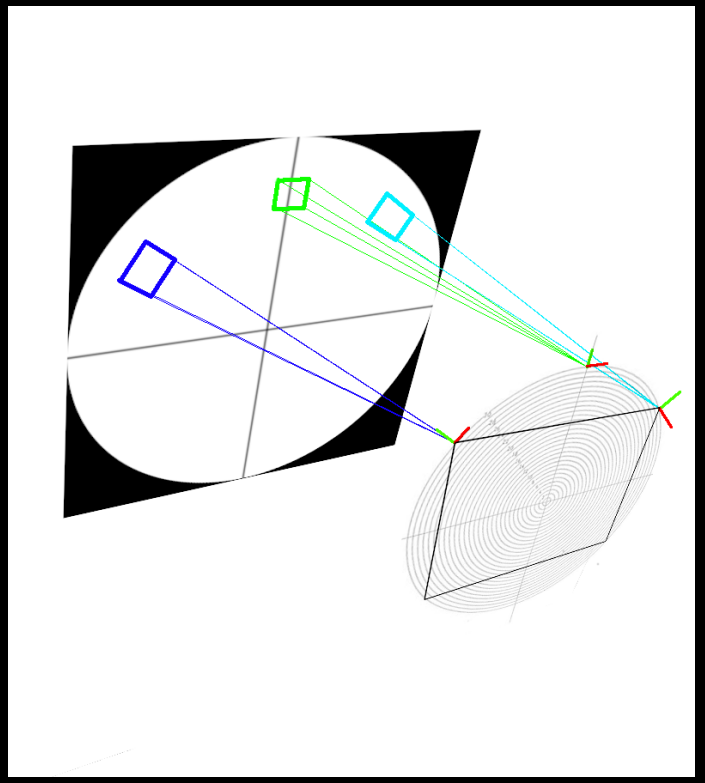
Precomputing the LUT

- How do we calculate these bounds, and how are they used to generate ray samples? How is the radial symmetry of the system harnessed?
- Precompute step is less than 10ms to fire "a few million" rays (your mileage may vary), which allows for UI interactivity of all system parameters
  - Each LUT entry's values are established starting at a point up the Y axis - this is the representative source point on the film for that LUT entry
  - The R2 low-discrepancy sequence is used to generate points in Normalized Rear Element Coords... sample rays are from the point on the film, to the point on the rear element, normalized
  - Trace each of these rays through the system, as described earlier, and break out of traversal if you violate any of the "valid ray" conditions
  - For any "valid ray" found to reach the front of the lens... it made it through the pupil, and we need to max() and min() the bounding box dimensions with this new, valid sample point in rear element space... the red and green frustums here show these bounds for two different LUT entries at the center and at the periphery of the film (this particular arrangement doesn't necessarily correspond to any specific lens)



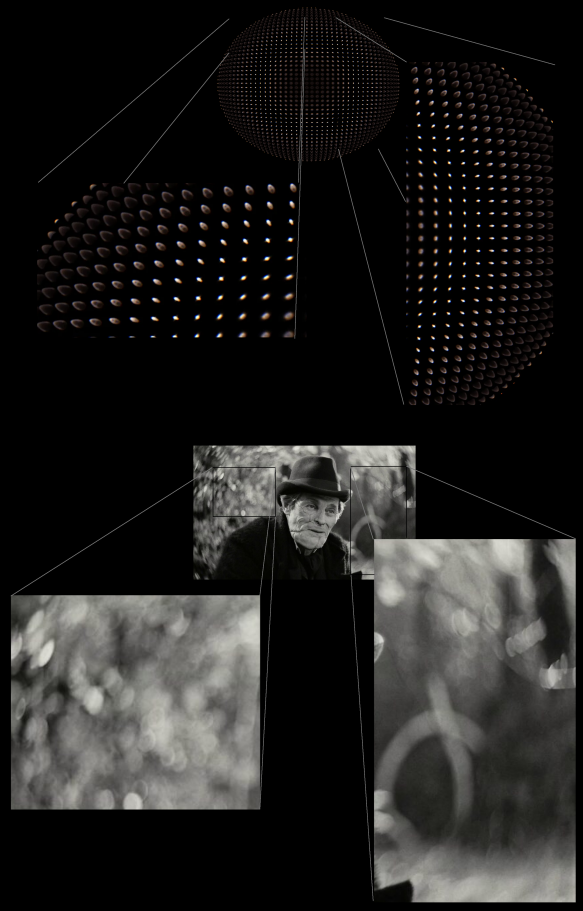
Off to the Ra(y)ces

- Generating ray samples at runtime
  - The 2D bounds we just established may seem unfriendly to radial symmetry. Using a simple 2D axis-aligned bounding box in this way, defined in terms of min and max on X and Y, it may not seem immediately intuitive how this is used for anything but points on the Y axis, like those that were used to generate the bounds...
  - I'm very pleased with the scheme I found here - it has a minor special case at the center of the image, where you need to just use constant X and Y values, due to a zero displacement vector from the pixel to the center of the image - you just need to establish a 2D basis that encodes the rotation...
  - The Y component of this basis is the pixel's displacement vector from the center, normalized, and you can construct the X component very cheaply from the .yx swizzle of the Y component - you can see the blue and cyan copies using the data for the green LUT entry, by using modified basis vectors in this way



Discussion + Future Directions

- Petzval "Swirly Bokeh"
  - I tried this just to see what would happen - it's white, diffusely emissive points on a black plane, a certain distance in front of the lens
  - Notice how the defocus blur creates varying distortion across the lens field - this is the "swirly" property
  - Unfortunately, and I believe this to be a consequence of my uniform sampling as I mentioned before, I'm not resolving the bokeh patterns quite I was expecting to see, from pictures I've seen (here from Poor Things)
  - I think at least part of this is from having more complex scene illumination coming into that lens, but an important aspect of the pupil sampling that is missing right now is the ability to sample nonuniform pupil radiance - due to the various distortions present in these systems, I think this is actually more the rule than the exception
- My plan going forward is to accumulate a tally of photons for each LUT entry as a texture, spanning across the Normalized Rear Element Coords - accumulating this under uniform illumination outside the lens (with the rays traced in the opposite direction) will give me the information I need to importance sample the pupil radiance... it may even make sense to add another dimension to that table, with respect to wavelength, as this variation can also cause significant changes in ray behavior



Thanks - Questions?